Big Data Lab

Lab 5: Spark SQL, Window Functions & Performance Tuning

Dr. Innocent Nyalala

IIT Madras Zanzibar
Department of Data Science and AI

Monday, 20 April 2026

Session Plan & Learning Objectives (Master's level)

Theory (1 hour)

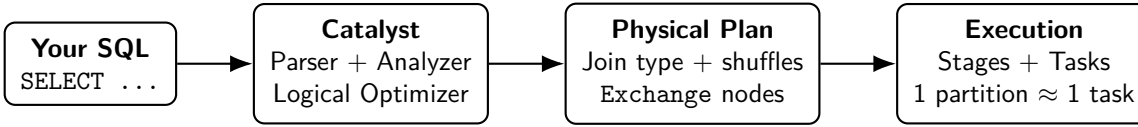
- Understand what Spark SQL is (DataFrame API + SQL + Catalyst + Tungsten)
- Learn the execution model: **logical plan** → **physical plan** → **stages/tasks**
- Interpret `explain(mode="formatted")` and identify the expensive operators
- Know what triggers a shuffle (`Exchange`) and why it dominates runtime
- Understand the main optimisation levers: partitioning, join strategy, AQE, caching

Practical (1 hour)

- Write Spark SQL queries end-to-end: select, filter, group, join
- Use CTEs (`WITH`) to structure multi-step queries clearly
- Use window functions for analytics without collapsing rows
- Profile a slow query using Spark UI + `explain()`
- Apply 2–3 fixes and measure speedup (before/after)

Outcome: you should be able to explain *why* a query is slow in Spark and choose the highest-impact fix.

How Spark Runs SQL



Rule of thumb: optimise the physical plan — especially reduce `Exchange` operators (shuffles).

Spark SQL Glossary (1/2): Data + Execution

Parallelism

- **Partition:** chunk of data processed by **one task**.
- **Task:** one unit of work on one core (processes one partition).

Data movement

- **Shuffle / Exchange:** re-partition across executors (network + disk).

DAG structure

- **Stage:** tasks between two shuffle boundaries.

Join strategy term

- **Broadcast join:** copy a *small* table to all executors to avoid shuffling the big table.

Spark SQL Glossary (2/2): Plans + Optimisation

Plans

- **Logical plan:** what the query means.
- **Physical plan:** how Spark will run it (joins, shuffles, scans).

AQE

- **AQE:** runtime re-optimisation using real shuffle statistics.

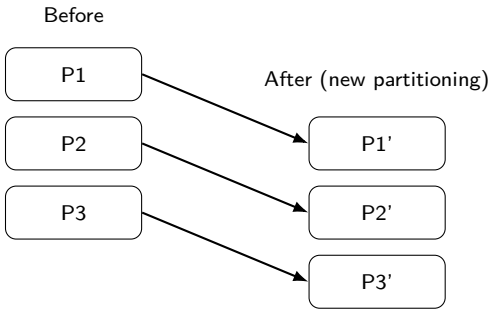
Where you see these

- `explain("formatted")` shows **Exchange**, join type, **AdaptiveSparkPlan**.
- Spark UI shows stages/tasks + shuffle read/write + skew.

What a Shuffle (`Exchange`) Is — and Why It's Expensive

A shuffle happens when Spark must re-partition data across the cluster.

- Data moves across the network between executors
- Spark writes/reads intermediate shuffle files (disk + network + CPU)
- Skew makes it worse: one huge partition ⇒ one long *straggler* task



Common shuffle triggers:

- `GROUP BY`, `DISTINCT`, `ORDER BY`
- Joins when data is not co-partitioned on the join key
- `repartition()` (always shuffles)

SQL From Zero (1/3): SELECT + WHERE (Social Media Example)

Scenario: a table `posts` with columns `post_id`, `user_id`, `country`, `likes`, `created_at`.

Goal: show Kenyan posts with $> 1{,}000$ likes

- `SELECT` chooses columns (projection)
- `WHERE` filters rows (predicate)

```
# SQL in Spark
spark.sql("""
SELECT post_id, user_id, likes, created_at
FROM posts
WHERE country = 'Kenya' AND likes > 1000
ORDER BY likes DESC
LIMIT 20
""").show(truncate=False)
```

In easy words

- Start with all posts
- Keep only Kenya + popular posts
- Show just the columns we care about

SQL From Zero (2/3): GROUP BY (Counts & averages)

Goal: average likes per country (who gets engagement?)

- `GROUP BY` forms groups
- Aggregates (`COUNT`, `AVG`, `SUM`) compute per-group values

```
spark.sql("""
SELECT country,
       COUNT(*)             AS n_posts,
       ROUND(AVG(likes),1) AS avg_likes,
       MAX(likes)           AS max_likes
FROM posts
GROUP BY country
ORDER BY avg_likes DESC
""").show()
```

Common pitfall

- Every selected column must be either:
  - in `GROUP BY`, or
  - aggregated (`AVG`, `MAX`, ...)

SQL From Zero (3/3): JOIN (Combine facts + metadata)

Scenario: `posts` + `users(user_id, age, verified)`

Goal: average likes by verification status

```
spark.sql("""
SELECT u.verified,
       COUNT(*)             AS n_posts,
       ROUND(AVG(p.likes),1) AS avg_likes
FROM posts p
JOIN users u
  ON p.user_id = u.user_id
GROUP BY u.verified
ORDER BY avg_likes DESC
""").show()
```

In easy words

- JOIN matches rows using a key
- Think: "lookup extra columns"
- Joins can trigger shuffles (expensive)

Spark SQL Pipeline

What happens in each box

- **APIs:** parse SQL / build DataFrame ops.
- **Catalyst:** resolves columns & types, rewrites the logical plan, chooses join strategies.
- **Physical plan:** concrete operators (scan, filter, join, `Exchange`).
- **Execution:** splits into stages; runs tasks on partitions.

Where it runs (Docker mental model)

- **Driver** = your notebook / `spark-submit` container (builds the plan).
- **Executors** = worker containers (run tasks).
- **Spark UI** shows what happened: stages/tasks/shuffles.
- In this lab: Spark master UI `:8080`, app UI `:4040`.

Takeaway: SQL is a *front-end*. Performance depends on the physical plan Spark generates.

Spark SQL: What You Can Express in SQL

Core query building blocks

- Filtering & projection: `SELECT`, `WHERE`
- Aggregation: `GROUP BY`, `HAVING`
- Joins: inner/left/right/full, semi/anti joins
- Sorting/limiting: `ORDER BY`, `LIMIT`

Analytical SQL (what makes Spark SQL powerful)

- CTEs: `WITH ...` (readable multi-step queries)
- Window functions: ranking, running totals, rolling metrics
- Nested data: `EXPLODE` via `LATERAL VIEW`
- Reshaping: `PIVOT`/`UNPIVOT`

Key idea

- Spark SQL is not "just SQL".
- It is SQL *compiled* into distributed execution.
- The same engine runs SQL and DataFrames.

SQL vs DataFrame API (Same Engine, Different Syntax)

Both compile to the same physical plan via Catalyst. Choose the interface that reduces bugs and improves readability.

Prefer SQL when:

- You need complex multi-join queries with many predicates
- The logic is declarative (reporting, aggregates, analytics)
- Your team reviews/maintains SQL comfortably

Prefer DataFrames when:

- You need programmatic control (loops, branches, functions)
- You are building reusable pipelines with tests
- You want type safety (Scala) or IDE help

CTEs (1/2): Break a Query into Named Steps

- A **CTE** (`WITH ...`) is a named subquery.
- Use it to make multi-step SQL readable.
- Spark typically inlines/optimises CTEs (you use them for clarity).

```
# Top-3 countries per month by revenue
spark.sql("""
WITH monthly AS (
  SELECT DATE_FORMAT(order_date,'yyyy-MM') AS month,
         country, SUM(order_total) AS revenue
  FROM orders_typed
  WHERE status='completed'
  GROUP BY month, country
), ranked AS (
  SELECT month, country, revenue,
         RANK() OVER (PARTITION BY month ORDER BY revenue DESC) AS rnk
  FROM monthly
)
SELECT month, country, revenue
FROM ranked
WHERE rnk <= 3
```

CTEs (2/2): Multiple CTEs + Global Comparison

```
# Compare each country's total vs global average
spark.sql("""
WITH clean AS (
  SELECT country, amount
  FROM orders_typed
  WHERE amount > 0
), totals AS (
  SELECT country, SUM(amount) AS total
  FROM clean
  GROUP BY country
), global_avg AS (
  SELECT AVG(total) AS avg_total FROM totals
)
SELECT t.country, t.total,
       ROUND(t.total / g.avg_total, 2) AS vs_avg
FROM totals t CROSS JOIN global_avg g
ORDER BY t.total DESC
""").show(truncate=False)
```

Lateral Views (1/2): EXPLODE Arrays into Rows

- `EXPLODE(array)` turns one row with an array into **many rows** (one per element).
- `LATERAL VIEW` "joins" those generated rows back to the original row.

```
orders_arr = spark.createDataFrame([
  (1,"Alice",["Electronics","Gadgets"]),
  (2,"Bob",  ["Furniture","Office"])
], ["order_id","customer","categories"])
orders_arr.createOrReplaceTempView("orders_array")

spark.sql("""
SELECT order_id, customer, category
FROM orders_array
LATERAL VIEW EXPLODE(categories) t AS category
""").show(truncate=False)
```

Lateral Views (2/2): OUTER EXPLODE + POSEXPLODE

- `OUTER` keeps the input row even if the array is `NULL` / empty (you get a row with `NULL` for the exploded value).
- `POSEXPLODE` also returns the element index (position) alongside the value.

```
# OUTER keeps rows when array is NULL/empty
spark.sql("""
SELECT order_id, customer, category
FROM orders_array
LATERAL VIEW OUTER EXPLODE(categories) t AS category
""").show(truncate=False)

# POSEXPLODE returns (pos, value)
spark.sql("""
SELECT order_id, pos, category
FROM orders_array
LATERAL VIEW POSEXPLODE(categories) t AS pos, category
""").show(truncate=False)
```

PIVOT (1/2): Rows →Columns

- `PIVOT` takes **distinct values in a column** (e.g., `country`) and turns them into **new columns**.
- You must pair it with an **aggregation** (e.g., `SUM(revenue)`), because many rows map into one output row.

```
spark.sql("""
CREATE OR REPLACE TEMP VIEW sales AS
SELECT * FROM VALUES
  ('Jan','Kenya',4500.0),('Jan','Tanzania',3200.0),
  ('Feb','Kenya',5100.0),('Feb','Tanzania',3800.0)
AS t(month, country, revenue)
""")

spark.sql("""
SELECT * FROM (SELECT month, country, revenue FROM sales)
PIVOT (SUM(revenue) AS total FOR country IN ('Kenya','Tanzania'))
ORDER BY month
""").show(truncate=False)
```

UNPIVOT (2/2): Columns →Rows

- UNPIVOT is the inverse of pivot: it turns **multiple columns** (e.g., Kenya, Tanzania) into **rows**.
- Useful when you want a "tidy" long table: (month, country, revenue).

```
# Spark 3.4+ UNPIVOT syntax
spark.sql("""
SELECT * FROM (
  SELECT month, Kenya, Tanzania FROM (
    SELECT month, country, revenue FROM sales
  ) PIVOT (SUM(revenue) FOR country IN ('Kenya','Tanzania'))
)
UNPIVOT (revenue FOR country IN (Kenya, Tanzania))
ORDER BY month, country
""").show(truncate=False)
```

Window Functions: One Practical Example

- Use windows when you need **group-aware** calculations without collapsing rows.
- Example: 7-day rolling engagement per user.

```
spark.sql("""
SELECT user_id, day, likes,
       AVG(likes) OVER (
         PARTITION BY user_id ORDER BY day
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS likes_avg_7d
FROM daily_likes
""").show()
```

Window Functions: Cheat Sheet (Core Concepts)

Syntax pattern (always)

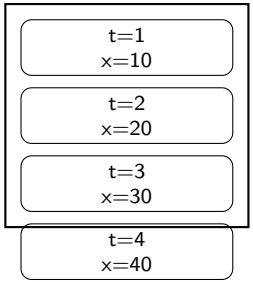
```
<func>(...)  OVER (PARTITION BY ...  ORDER BY ...
                   [frame])
```

- **PARTITION BY** = groups (does *not* reduce rows)
- **ORDER BY** = sequence inside each group
- **Frame** = which neighbouring rows are included

Key distinction

- `GROUP BY` *reduces rows* (one row per group)
- Window functions *add columns* (keep original rows)

Frame example
(last 3 rows)



Window Functions: Quick Recipes

Goal	Pattern
Top-k per group	`ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC)` then filter `<= k`
Running total	`SUM(x) OVER (PARTITION BY g ORDER BY t ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
Rolling average	`AVG(x) OVER (PARTITION BY g ORDER BY t ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
Previous/next value	`LAG(x,1)/LEAD(x,1) OVER (PARTITION BY g ORDER BY t)`

Mini example (social media): 7-day rolling likes per user

```
spark.sql("""
SELECT user_id, day, likes,
       AVG(likes) OVER (
         PARTITION BY user_id ORDER BY day
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS likes_avg_7d
FROM daily_likes
""").show()
```

Window Frames (1/2): ROWS vs RANGE in Plain Words

- A **frame** tells Spark which rows are "visible" to the window function.
- **ROWS BETWEEN** counts *physical rows* (e.g., last 7 rows).
- **RANGE BETWEEN** counts a *value range* in the ORDER BY column.
- If there are ties in the ORDER BY value: RANGE may include more rows.

Rule of thumb: for time-series rolling metrics, `ROWS BETWEEN` is usually the clearest.

Window Frames (2/2): Short SQL Example

```
# rolling avg (ROWS) vs logical range (RANGE)
spark.sql("""
SELECT country, month, revenue,
  ROUND(AVG(revenue) OVER (
    PARTITION BY country ORDER BY month
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ),0) AS rolling_rows,
  ROUND(AVG(revenue) OVER (
    PARTITION BY country ORDER BY month
    RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
  ),0) AS rolling_range
FROM sales
ORDER BY country, month
""").show()
```

Advanced Windows (1/2): NTILE + FIRST/LAST VALUE

```
# NTILE: quantile buckets (e.g., top 10%)
spark.sql("""
SELECT order_id, country, order_total,
       NTILE(10) OVER (ORDER BY order_total DESC) AS decile
FROM orders_typed
""").show(10)

# FIRST/LAST within a partition
spark.sql("""
SELECT country, month, revenue,
  FIRST_VALUE(revenue) OVER (
    PARTITION BY country ORDER BY month
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS first_rev,
  LAST_VALUE(revenue) OVER (
    PARTITION BY country ORDER BY month
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS last_rev
FROM sales
""").show()
```

Advanced Windows (2/2): QUALIFY (or Filter) Top-k

```
# Spark 3.5+: QUALIFY filters after window evaluation
spark.sql("""
SELECT order_id, country, order_total,
       RANK() OVER (PARTITION BY country ORDER BY order_total DESC) AS rnk
FROM orders_typed
QUALIFY rnk <= 3
""").show()

# Older Spark: SELECT * FROM (...) WHERE rnk <= 3
```

Partitioning: The #1 Performance Lever

Why partition count matters

- Spark executes **1 task per partition** (parallelism comes from partitions)
- Too few partitions ⇒ cores idle; large partitions may OOM
- Too many partitions ⇒ overhead + many small shuffle files
- Skew: one huge partition ⇒ one long *straggler* task

Rule of thumb: aim for **128–200 MB** per partition for shuffle-heavy stages.

Tuning rules (quick table)

Symptom	Action
Too few tasks	`repartition(N)`
Too many tiny tasks	`coalesce(N)`
Hot key skew	Salt key or broadcast small side
After big filter	Repartition again (avoid skew)
Before single output file	`coalesce(1)` (careful!)

AQE note: with `spark.sql.adaptive.enabled=true`, Spark can coalesce post-shuffle partitions automatically.

repartition() vs coalesce(): What Changes?

repartition(N)

- **Forces a shuffle** (Exchange)
- Use to **increase** partitions or **balance skew**

coalesce(N)

- **No shuffle** (merges partitions locally)
- Use to **decrease** partitions (fewer output files)

Rule

- Increase/balance ⇒ `repartition`
- Decrease ⇒ `coalesce`

Watch for

- If you see `Exchange` after `repartition`: that's expected.

repartition() vs coalesce(): Minimal Demo

```
df = spark.read.parquet("s3a://warehouse/silver/orders/")
print(df.rdd.getNumPartitions())

# shuffle (balances)
df2 = df.repartition(20)

# no shuffle (merge)
df3 = df.coalesce(4)
```

Bucketing (1/2): Write Bucketed Tables

```
# Bucket both tables on the join key with the SAME bucket count
spark.table("orders_typed").write \
  .bucketBy(8, "country").sortBy("country") \
  .mode("overwrite").saveAsTable("orders_bucketed")

spark.read.parquet("s3a://warehouse/raw/country_meta.parquet").write \
  .bucketBy(8, "country").sortBy("country") \
  .mode("overwrite").saveAsTable("country_meta_bucketed")
```

Bucketing (2/2): Join (Look for NO Exchange)

```
spark.sql("""
SELECT o.country, c.region, COUNT(*) AS n_orders
FROM orders_bucketed o
JOIN country_meta_bucketed c
  ON o.country = c.country
GROUP BY o.country, c.region
""").explain("formatted")

# In the plan: fewer/zero Exchange nodes => less shuffle
```

Caching: When It Helps (and When It Hurts)

- Cache when you reuse the same DataFrame **2+ times**.
- Caching can hurt if memory is tight (GC/spills) or you cache too much.
- Always `unpersist()` when finished.

Default: `cache()` = `persist(MEMORY_AND_DISK)`.

Caching: Minimal Example + Storage Levels

```python
from pyspark import StorageLevel

df = spark.read.parquet("s3a://warehouse/silver/orders/")

# safe default
cached = df.cache(); cached.count()

# explicit control
cached.persist(StorageLevel.MEMORY_AND_DISK)

# free memory when done
cached.unpersist()
```

AQE (Adaptive Query Execution): What It Does and Why It Matters

Definition

- AQE re-optimises parts of the physical plan **at runtime**
- It uses **actual** shuffle statistics (row counts, sizes)
- Goal: reduce shuffle cost and choose better join strategies

Enable / check

- Enable: `spark.sql.adaptive.enabled=true`
- Check in the plan: look for `AdaptiveSparkPlan` / nodes marked `(AQE)`

3 high-impact AQE optimisations

1. **Coalesce shuffle partitions:** merges many tiny partitions after a shuffle
2. **Join strategy switch:** SMJ → broadcast if one side becomes small
3. **Skew handling:** splits skewed partitions into smaller chunks

AQE in Action: Enable and Observe

```
import time

# Disable AQE -- baseline (manual shuffle partitions = 200)
spark.conf.set("spark.sql.adaptive.enabled", "false")
spark.conf.set("spark.sql.shuffle.partitions", "200")

t0 = time.time()
spark.sql("""
    SELECT country, category, SUM(order_total) AS total
    FROM orders_typed
    GROUP BY country, category
    ORDER BY total DESC
""").collect()
print(f"Without AQE: {time.time()-t0:.2f}s   (200 tiny shuffle partitions)")

# Enable AQE -- Spark merges tiny partitions automatically
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "128m")

t0 = time.time()
spark.sql("""
```

Data Skew in Big Data: What It Is (and Why Jobs "Hang")

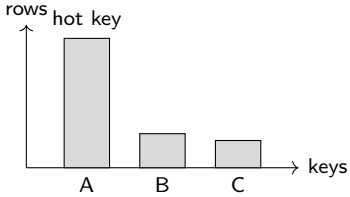
Definition (easy words)

- **Skew** means the data is **not evenly distributed** across keys.
- Example: in a join on `country`, one country has 95% of the rows.
- Spark parallelism is per-partition, so skew creates **one huge partition**.

Why it hurts

- 1 partition $\approx$ 1 task $\Rightarrow$ one *straggler* task determines total runtime
- The straggler often shows high shuffle read + high GC time

Skew picture: one hot key



How to Detect Data Skew (UI + Code)

Spark UI symptoms

- Stage: most tasks finish fast, 1–2 take 10–100× longer
- One task has much larger **Shuffle Read** than the median
- The slow task may show high **GC time** / spills

Quick code check: key frequency (example: `country`)

```
# look for a dominant key ("hot" value)
skewed.groupBy("country").count() \
  .orderBy(desc("count")).show(10)

# for joins: check both sides of the join key
posts.groupBy("user_id").count().orderBy(desc("count")).show(10)
users.groupBy("user_id").count().orderBy(desc("count")).show(10)
```

Rule of thumb

- If top-1 key $>$ 50% of rows, expect skew problems.
- If the join key contains many NULLs, expect skew.

Where skew appears

- `GROUP BY`, `JOIN`, `DISTINCT`

How to Fix Skew: Techniques (When to Use Which)

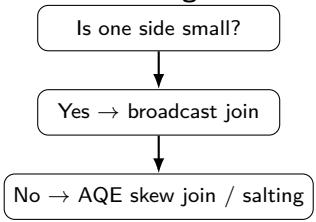
Automatic (Spark 3+)

- **AQE skew join handling:** Spark splits skewed shuffle partitions
- Works best when AQE is enabled and stats are available

Manual techniques

- **Salting:** add random salt to spread the hot key
- **Split hot key:** process hot key separately, then `union`
- **Broadcast:** broadcast small dimension table to avoid shuffle join

Decision guide



Salting: Why It Works (Picture)

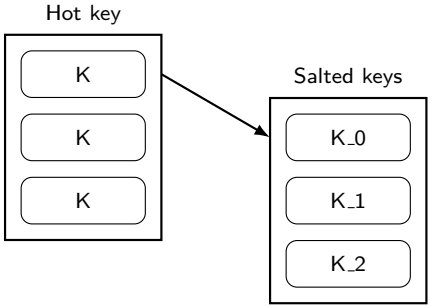
Idea

- Replace one hot key `K` with many keys: `K_0`, `K_1`, `...`, `K_S-1`
- This spreads the work across **S partitions** (instead of 1)
- Then you aggregate/join in **two phases** and remove the salt

When to use

- A single key dominates a `GROUP BY` or join
- Broadcasting is not possible (both sides are large)

Before vs after



Skew Demo + Salting Fix (Country Example)

```
import time
from pyspark.sql.functions import col, lit, when, rand, desc, sum, concat

# Create skewed data (90% Kenya)
skewed = spark.range(0, 2_000_000) \
  .withColumn("country",
    when(col("id") % 10 < 9, lit("Kenya")).otherwise(lit("Tanzania"))) \
  .withColumn("amount", (rand() * 500).cast("double"))

print("=== Key distribution ===")
skewed.groupBy("country").count().orderBy(desc("count")).show()

# Skewed groupBy (AQE off)
spark.conf.set("spark.sql.adaptive.enabled", "false")
t0 = time.time()
skewed.groupBy("country").agg(sum("amount").alias("total")).collect()
t_skewed = time.time() - t0
print(f"Skewed groupBy: {t_skewed:.2f}s")

# SALTING FIX (two-phase aggregation)
SALT_FACTOR = 20
salted = skewed.withColumn("salt", (rand()*SALT_FACTOR).cast("int")) \
```

Other Skew Fixes: Broadcast and Split-Hot-Key

Broadcast join (avoid shuffle)

```
from pyspark.sql.functions import broadcast

# example: users is small, posts is large
joined = posts.join(broadcast(users), "user_id")
```

When: one side is small enough to fit in executor memory.

Split hot key (process separately)

```
# example: one country dominates
hot  = orders.filter(col("country") ==
     "Kenya")
cold = orders.filter(col("country") !=
     "Kenya")

res = hot.groupBy("country").agg(...) \

     .unionByName(cold.groupBy("country").agg(.
```

When: one key dominates and needs special treatment.

Join Hints (1/2): Broadcast vs Sort-Merge

```
# 1) BROADCAST hint: force broadcast of the smaller table
spark.sql("""
  SELECT /*+ BROADCAST(c) */ o.order_id, c.region, o.order_total
  FROM orders_typed o
  JOIN country_meta c ON o.country = c.country
""").explain(mode="simple")  # -> BroadcastHashJoin

# 2) MERGE hint: force Sort-Merge Join
spark.sql("""
  SELECT /*+ MERGE(o, c) */ o.country, SUM(o.order_total) AS total
  FROM orders_typed o
  JOIN country_meta c ON o.country = c.country
  GROUP BY o.country
""").explain(mode="simple")  # -> SortMergeJoin
```

Join Hints (2/2): Shuffle Hash + Partition Hints

```
# 3) SHUFFLE_HASH: Shuffle Hash Join (often good for medium data)
spark.sql("""
  SELECT /*+ SHUFFLE_HASH(o) */ *
  FROM orders_typed o JOIN country_meta c
  ON o.country = c.country
""").explain(mode="simple")

# 4) Partition hints (force shuffle partitioning)
spark.sql("SELECT /*+ REPARTITION(20) */ * FROM orders_typed").explain()
spark.sql("SELECT /*+ COALESCE(4) */ * FROM orders_typed").explain()

# DataFrame API hint:
df_orders.hint("broadcast").join(country_meta, "country")
```

Benchmarking (1/2): Define the Slow Query + Baseline

```
import time

spark.conf.set("spark.sql.adaptive.enabled", "false")
spark.conf.set("spark.sql.shuffle.partitions", "200")

def run_query():
  return spark.sql("""
    SELECT o.country, c.region, DATE_FORMAT(o.order_date,'yyyy-MM') AS month,
           COUNT(*)           AS n_orders,
           SUM(o.order_total) AS total_revenue,
           AVG(o.order_total) AS avg_order
    FROM orders_typed o
    JOIN country_meta c ON o.country = c.country
    GROUP BY o.country, c.region, month
    ORDER BY total_revenue DESC
  """).collect()

t0 = time.time(); run_query(); baseline = time.time() - t0
print(f"Baseline: {baseline:.2f}s")
```

Benchmarking (2/2): Apply Fixes + Compare Speedup

```
# Fix 1: fewer shuffle partitions
spark.conf.set("spark.sql.shuffle.partitions", "8")
t0 = time.time(); run_query(); t1 = time.time() - t0
print(f"shuffle.partitions=8: {t1:.2f}s  ({baseline/t1:.1f}x)")

# Fix 2: enable AQE + cache hot table
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.table("orders_typed").cache().count()

t0 = time.time(); run_query(); t2 = time.time() - t0
print(f"AQE + cache: {t2:.2f}s  ({baseline/t2:.1f}x)")
```

Reading the Spark UI Like a Pro

SQL / DataFrame tab (per query)

- Physical-plan DAG + timings per operator
- Click nodes to see rows in/out, spill, shuffle size
- Learn to spot the expensive operators:
  - **Exchange** = shuffle (network + disk)
  - **BroadcastHashJoin** = fast join (when one side is small)
  - **FileScan** + **Filter** below it = predicate pushdown
- Nodes marked **(AQE)** indicate runtime re-optimisation

Exercise: bring your slowest query, find the first **Exchange**, then apply one fix and measure speedup.

Stages tab (per stage)

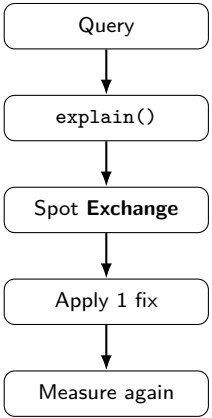
Signal	Interpretation / action
Shuffle Read large	Reduce shuffle partitions; broadcast; bucket
GC Time high	Memory pressure; cache carefully; repartition
Task time spread	Skew; salt; AQE skew join
Spill to disk	Too-large partitions; increase memory; repartition
Rows in $\gg$ out	Pushdown working; pruning likely effective

URLs: Master UI `localhost:8080` SQL UI `localhost:4040`

Performance Tuning Workflow (What to Do Every Time)

Step-by-step

1. Write the query for correctness first (small sample + `LIMIT`)
2. Inspect the plan: `df.explain("formatted")` or SQL `EXPLAIN FORMATTED`
3. Find the cost drivers: **Exchange** (shuffle), sort, wide joins
4. Change *one* lever (broadcast / partitions / rewrite / AQE) and re-measure



Easy words (for non-SQL background)

- **Logical plan:** what you asked for (relational algebra)
- **Physical plan:** how Spark will actually run it (joins, shuffles, scans)

Baseline Configs + A Tiny Tuning Example

Baseline lab configs (start here)

Config	Value
spark.sql.shuffle.partitions	8--20
spark.sql.adaptive.enabled	true
spark.sql.autoBroadcastJoinThreshold	10m

Principle: measure baseline → change one thing → measure again.

Example: fix an unnecessary shuffle

```
q = """SELECT country, COUNT(*) n
       FROM orders_typed
       GROUP BY country"""

spark.sql(q).explain("formatted")
# If you see Exchange -> shuffle

spark.conf.set("spark.sql.shuffle.partitions","
# run again and compare time
```

Common Spark SQL Mistakes (and Better Patterns)

Mistakes that hurt performance

- `SELECT *` in subqueries: disables column pruning
- `DISTINCT` after `GROUP BY`: redundant shuffle
- `ORDER BY` without `LIMIT`: global sort of all data
- **UDFs in filters**: blocks pushdown/codegen
- **Join predicates with OR**: often prevents optimisations
- **Wide rows**: select fewer columns before joins/aggregations

Better patterns (quick wins)

- Select only needed columns early
- Prefer built-in functions (dates, strings) over UDFs
- Use `COALESCE` for NULL-safe aggregates
- Use CTEs (`WITH`) to make steps explicit and debuggable
- Use window functions for within-group ranking instead of global sorts
- During exploration: `TABLESAMPLE` + `LIMIT`

Debug habit: after each rewrite, compare physical plans and count `Exchange` nodes.

Assignment 5: Spark SQL + Performance Tuning (100 marks)

1. **(25) Window functions:** using a time-series dataset, implement:
   - 7-day rolling average
   - month-over-month growth rate
   - top-3 per group using `RANK`
   - `FIRST_VALUE` and `LAST_VALUE` per group

   Explain each function's purpose (**3 sentences each**) and show output.

2. **(25) Performance tuning:** take a slow query (join + groupBy + orderBy). Measure baseline time. Apply $\geq$**3 optimisations** (partitioning, caching, AQE, broadcast, bucketing). Report speedup and include before/after Spark UI evidence.

Week 5 Assignment (Part 2)

3. **(25) Data skew:** create a dataset with >90% of data in one key. Screenshot Spark UI showing uneven task times. Apply salting and show improvement.
4. **(20) Bucketing:** write a table bucketed by a join key and join it with another bucketed table. Show `explain()` with and without bucketing and report speedup.
5. **(5) Reflection:** when should you use `repartition()` vs `coalesce()`? Give one production scenario for each (4–6 sentences).

Next Week — Lab 6: Data Orchestration with Apache Airflow

What we will cover

- Why orchestration exists (pipelines, dependencies, retries)
- Airflow concepts: DAGs, operators, sensors
- Writing your first DAG
- Triggering Spark jobs (`SparkSubmitOperator`)
- Scheduling, retries, alerts
- Monitoring + debugging in Airflow UI
- Brief look at Dagster

Pre-lab setup

- Airflow UI: `http://localhost:8090`
- Login: `admin` / `admin`

If it is not running:

```
docker compose ps | grep airflow
```

```
docker compose up -d
    airflow-webserver
    airflow-scheduler
```